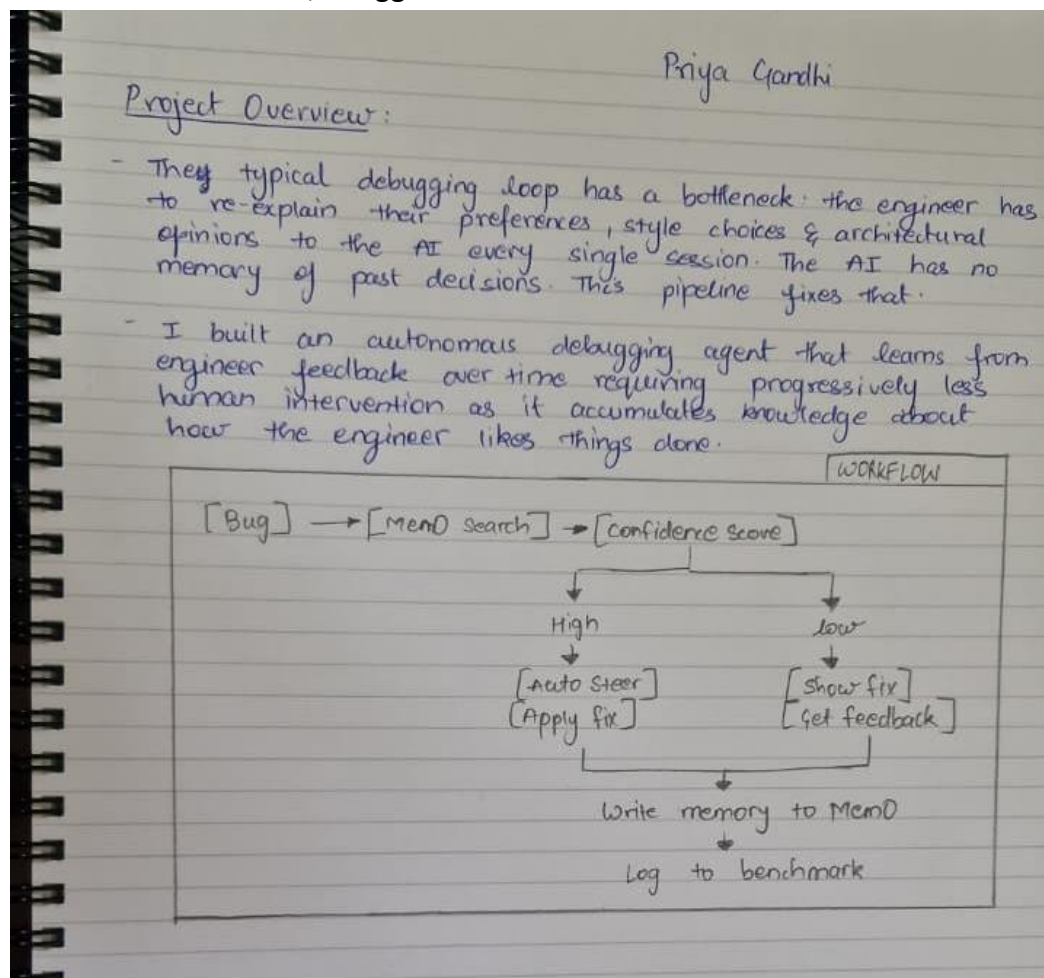


Report -

Explain your project and how the pipeline works end to end

A buggy Python file is loaded as **input**. Before proposing any fix, the **agent searches Mem0** for relevant memories from past sessions, stored preferences, patterns, things to avoid. From those memories it computes a **confidence score** by averaging the similarity scores Mem0 returns. If confidence crosses the configured **threshold**, the agent takes the **autonomous path**: it reads the memories, derives the fix direction the engineer would have given, and generates a fix entirely on its own without asking anything. If confidence is below the threshold, typically in early sessions when **few memories exist**, it takes the **human path**: it generates a fix using whatever memories it has as context, shows it to the engineer, and asks for feedback.

Feedback is where learning happens. **Accept** stores the successful approach. **Reject** stores what to avoid and why. **Edit** triggers a principle extraction step, a short LLM call that derives the underlying preference from the engineer's correction, something like "developer prefers specific exceptions with logging over bare except clauses." That principle gets stored in Mem0. **Steer** stores the engineer's redirect as a preference for that bug type. Every session, autonomous or human, is logged to a **benchmark file**.



Design Choices

Why did I implement it this way?

The core **problem** with regular AI coding tools is that they operate within a **fixed context window** and have **no memory across sessions**. Every new chat starts from zero and the engineer has to re-explain their preferences, past decisions, and architectural opinions every single time.

The **solution** needed a **persistent memory layer** that could survive **across sessions**, store structured preferences, and be flexible enough to **update or delete outdated information**. I chose **Mem0** for this. It sits between the agent and the vector store, handles semantic search over stored memories, and gives fine-grained control over what gets stored and retrieved. **Qdrant** is used as the vector store backend, running on the cloud for free.

The **confidence layer** sits on top of Mem0's similarity scores. When the agent retrieves memories, each one comes back with a **relevance score between 0 and 1**. I average these scores and apply a boost if multiple strong memories agree. If the result crosses the threshold, the agent **acts autonomously**, no human input needed. This is the mechanism that **directly reduces intervention over time**.

Where did I have to make tradeoff decisions?

Mem0 LLM extraction vs infer=False - Mem0 by default uses an LLM internally to extract and deduplicate memories before storing them. This produced cleaner memories but consumed 8,000+ tokens per write call, which immediately exhausted Groq's free tier limit of 6,000 tokens per minute. I switched to `infer=False` which stores memories directly without internal LLM processing. To compensate for the loss of intelligent extraction, I added my own `extract_principle` step, a short focused Groq call that extracts the underlying preference from an engineer's edit before storing it. This gives me the extraction quality I need without the token cost.

Model choice - I used llama-3.3-70b-versatile on Groq's free tier for fix generation and principle extraction. The free tier has a 100,000 token daily limit which constrained how many benchmark rounds I could run in a single day.

Benchmarking Methodology

To prove the pipeline genuinely improves over time rather than just getting easier bugs, I built a controlled benchmark using a fixed set of **10 canonical bug files** covering common Python bug types (null checks, return type errors, exception handling, off-by-one errors, logic errors, and edge cases). These files never change between rounds. The agent runs through all 10 bugs in Round 1, Round 2, and Round 3. Since the input is identical each round, any improvement in the metrics can only be explained by the memory accumulating, not by easier inputs.

Priya Gandhi
pgandhi4@ncsu.edu
Soren interview report

Four metrics are tracked per round: autonomy rate (what percentage of bugs went autonomous), acceptance rate (what percentage of fixes were accepted), average confidence score, and intervention depth (none / light / medium / heavy). The improvement across rounds on these metrics is the proof of learning.

Results

Confidence scores increased session over session as memories accumulated. Once the confidence threshold was met, the agent began routing bugs to the autonomous path and applying fixes without engineer input. I ran the benchmark with a lowered confidence threshold (**0.51**) to demonstrate the learning curve within the token constraints of the free tier. With more tokens available, the same improvement would be visible at the standard **0.80 threshold** over more rounds.

The benchmark tracks four metrics across rounds: **autonomy rate, acceptance rate, average confidence score, and intervention depth**. All four showed improvement between Round 1 and Round 2 on the same fixed set of 10 canonical bugs.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(venv) PS C:\Users\gandh\Documents\Soren> python main.py --mode report
Avg Confidence: 0.29
Heavy Interventions: 10.0%
Medium Interventions: 30.0%

IMPROVEMENT OVER TIME
-----
Band      Sessions  Autonomy%  Accept%    AvgConf    Heavy%
-----
1-3       3         0%         67%        0.319      0%
4-6       3         0%         67%        0.316      0%
7-10      4         0%         50%        0.249      25%

PER BUG TYPE BREAKDOWN
-----
Bug Type      Sessions  Autonomy%  Accept%
-----
edge_case      1         0%         0%
exception_handling  2         0%        100%
logic_error    1         0%         0%
null_check     2         0%         50%
off_by_one     1         0%         0%
return_type    2         0%        100%
type_error     1         0%        100%

SESSION LOG
-----
ID  Bug ID  Path  Action  Conf  Depth
---  ---  ---  ---  ---  ---
1   bug_01  human  a       0.389  light
2   bug_02  human  a       0.409  light
3   bug_03  human  e       0.159  medium
4   bug_04  human  a       0.409  light
5   bug_05  human  e       0.233  medium
6   bug_06  human  a       0.307  light
7   bug_07  human  r       0.311  heavy
8   bug_08  human  a       0.186  light
9   bug_09  human  e       0.202  medium
10  bug_10  human  a       0.299  light

=====
```

Priya Gandhi
pgandhi4@ncsu.edu
Soren interview report

Ps. I ran out of tokens when it began taking autonomous path. Following image shows increase of confidence score and agent choosing autonomous path. I would appreciate if you can consider the final result after token reset.

```
(venv) PS C:\Users\gandh\Documents\Soren> python main.py --mode canonical
Confidence threshold: 0.5
Developer ID: dev_01

Running canonical benchmark - 10 bugs

=====
Session 70 | Bug: bug_01 | Type: return_type
=====
Memories retrieved: 20 | MEDIUM (0.71) - showing fix, needs review
Routing to: AUTONOMOUS PATH
Traceback (most recent call last):
  File "C:\Users\gandh\Documents\Soren\main.py", line 181, in <module>
    run_canonical_benchmark()
  File "C:\Users\gandh\Documents\Soren\main.py", line 145, in run_canonical_benchmark
    run_debug_session()
  File "C:\Users\gandh\Documents\Soren\main.py", line 69, in run_debug_session
    steering, fix = autonomous_steering(bug_description, code, memories)
    ~~~~~
  File "C:\Users\gandh\Documents\Soren\agent\steering.py", line 46, in autonomous_steering
    steering_response = groq_client.chat.completions.create(
    ~~~~~
  File "C:\Users\gandh\Documents\Soren\venv\Lib\site-packages\groq\resources\chat\completions.py", line 461, in create
    return self._post(
    ~~~~~
  File "C:\Users\gandh\Documents\Soren\venv\Lib\site-packages\groq\base_client.py", line 1284, in post
    return cast(ResponseT, self.request(cast_to, opts, stream=stream, stream_cls=stream_cls))
    ~~~~~
  File "C:\Users\gandh\Documents\Soren\venv\Lib\site-packages\groq\base_client.py", line 1071, in request
    raise self._make_status_error_from_response(err.response) from None
groq.RateLimitError: Error code: 429 - {'error': {'message': 'Rate limit reached for model `llama-3.3-70b-versatile` in organization `org_01kpgm70mejgs6bwp065svdpv` service tier `on_demand` on tokens per day (TPD): Limit 100000, Used 99589, Requested 587. Please try again in 2m32.064s. Need more tokens? Upgrade to Dev Tier today at https://console.groq.com/settings/billing', 'type': 'tokens', 'code': 'rate_limit_exceeded'}}

(venv) PS C:\Users\gandh\Documents\Soren> python main.py --mode report
Confidence threshold: 0.5
Developer ID: dev_01

=====
DEBUG AGENT - BENCHMARK REPORT
=====

Total sessions logged: 69
=====

OVERALL STATS
-----
Autonomy Rate:      4.3%
Acceptance Rate:    84.1%
Avg Confidence:     0.413
Heavy Interventions: 2.9%
Medium Interventions: 13.0%

IMPROVEMENT OVER ROUNDS (same 10 bugs repeated)
-----
Round      Sessions  Autonomy%  Accept%    AvgConf    Heavy%
-----
Round 1 (1-10)  10        0%         80%        0.283      0%
Round 2 (11-20) 10        0%         90%        0.306      0%
Round 3 (21-30) 10        0%         70%        0.368      20%
Round 4 (31-40) 10        0%         80%        0.415      0%

IMPROVEMENT DELTA (round over round)
-----
Round 1 (1-10) → Round 2 (11-20)
Autonomy:    → +0.0%
Acceptance:  ↑ +10.0%
Confidence:  ↑ +0.023

Round 2 (11-20) → Round 3 (21-30)
Autonomy:    → +0.0%
Acceptance:  ↓ -20.0%
Confidence:  ↑ +0.062

Round 3 (21-30) → Round 4 (31-40)
Autonomy:    → +0.0%
Acceptance:  ↑ +10.0%
Confidence:  ↑ +0.047
```

How I Would Improve This Given More Time

1. Faster, more accurate confidence growth

Currently confidence grows gradually because memories are stored one at a time from individual sessions. A better approach would be to seed the memory store upfront and have

the engineer fill out a short preference questionnaire at the start, store those answers as structured memories, and give the agent a meaningful baseline from session one rather than starting completely cold.

2. Team-level memory with multiple developer IDs

Right now all memories are scoped to a single `developer_id`. In a team setting, different engineers have different preferences which can conflict. The improvement would be to support team-level shared memories (architectural decisions, codebase conventions) alongside individual memories (personal style preferences), and let the agent reason over both. Mem0 supports multiple scopes natively, this would be a configuration and retrieval logic change.

Other Observations

The most interesting design tension in this project is between autonomy and correctness. Lowering the confidence threshold increases the autonomy rate, which looks good on the benchmark, but risks the agent acting on weak or contradictory memories and producing wrong fixes without asking. The threshold is intentionally configurable for this reason. In a production setting you would want it high. For a demo or early adoption phase you would lower it to show the learning behavior more quickly. Knowing where to set it is itself a judgment call that depends on how much the engineer trusts the accumulated memory.